# Chapter 1 Solutions: Problems P31–P34

*Computer Networking: A Top-Down Approach*, 9th Edition (Kurose & Ross) — Chapter 1: Computer Networks and the Internet

---

## P31 — Message Segmentation Delay

Setup: message size M = $10^6$ bits. Every link in Figure 1.28 has rate R = 5 Mbps = $5\times10^6$ bps. There are two packet switches, so there are three links between source and destination (source→switch 1, switch 1→switch 2, switch 2→destination). Propagation, queuing, and processing delay are all ignored, so the only delay is transmission delay.

### (a) Without message segmentation

Time to move the whole message from the source host to the first switch is just the transmission delay of the entire message onto that link:

$$t = M / R = 10^6 \text{ bits} / (5\times10^6 \text{ bps}) = 0.2 \text{ s} = 200 \text{ ms}$$

Because each switch uses **store-and-forward** switching, it cannot forward a single bit until it has received the *entire* message. So the message is fully received, then fully re-sent, once per link, with no overlap between links. With 3 links, the total delay is 3 times the single-link transmission delay:

$$\text{Total} = 3 \times (M / R) = 3 \times 0.2 \text{ s} = 0.6 \text{ s} = 600 \text{ ms}$$

**Answer:** 200 ms to reach the first switch; 600 ms total, source host to destination host.

### (b) Segmented into 100 packets of 10,000 bits each

Each packet's transmission time on one link:

$$t_{pkt} = 10{,}000 \text{ bits} / (5\times10^6 \text{ bps}) = 0.002 \text{ s} = 2 \text{ ms}$$

So it takes 2 ms to move the *first* packet from the source host to the first switch.

For the second packet: the source host doesn't wait for anything — as soon as it finishes transmitting packet 1 (at t = 2 ms), it immediately starts transmitting packet 2 onto the same link (this is what makes pipelining possible, and it's exactly what's described in the problem: while packet 1 moves switch-1→switch-2, packet 2 moves source→switch-1). Packet 2 then takes another 2 ms to be fully clocked onto the link, so it is fully received at the first switch at:

$$t = 2 \times 2 \text{ ms} = 4 \text{ ms}$$

**Answer:** 2 ms for the first packet to reach switch 1; the second packet is fully received at switch 1 at t = 4 ms.

### (c) Total time with message segmentation, and comparison with (a)

With segmentation, packets are pipelined across the 3 links instead of the whole message being store-and-forwarded as one block. The last (100th) packet finishes leaving the source host at 100 × 2 ms = 200 ms (the source sends all 100 packets back-to-back with no gaps). From the moment it arrives at switch 1, it still has to cross the remaining 2 links (switch 1→switch 2 and switch 2→destination), each adding one more packet transmission time of 2 ms, since by then those links are already clear of earlier

packets:

$$\text{Total} = (N + k - 1) \times t_{pkt}, \quad N = 100 \text{ packets}, k = 3 \text{ links}$$

$$\text{Total} = (100 + 3 - 1) \times 2 \text{ ms} = 102 \times 2 \text{ ms} = 204 \text{ ms}$$

**Answer:** 204 ms, versus 600 ms without segmentation — segmentation is **almost 3× faster** here (600/204 ≈ 2.94). The gain comes entirely from pipelining: once segmented, a switch can start forwarding the earliest packets while later packets of the same message are still arriving, instead of sitting idle until the entire $10^6$-bit message has arrived.

### (d) Other reasons to use message segmentation (besides lower delay)

- **Cheaper error recovery.** If a bit error corrupts one packet, only that small packet needs to be retransmitted — not the entire original message.
- **Fairer sharing of links.** Small packets from many different senders can be interleaved (statistically multiplexed) on a shared link. A single giant unsegmented message would monopolize a link for a long stretch of time, delaying everyone else's traffic.
- **Lower memory/buffering requirements at switches.** A switch only ever needs to buffer one small packet at a time rather than an entire large message.

### (e) Drawbacks of message segmentation

- **Header overhead.** Every packet needs its own header (addressing, sequence numbers, checksums, etc.), so segmenting into many small packets means more total overhead bits relative to actual message data.
- **More processing.** Each packet has to be individually processed, routed, and queued by the end hosts and every switch/router along the path, instead of handling one large unit.
- **Reassembly complexity.** The destination must buffer packets, detect loss, and handle out-of-order arrival (packets can even take different paths), which the unsegmented case never has to deal with.

## P32 — Interactive Animation & Propagation Delay

This problem asks you to run the book's own “Message Segmentation” Flash/HTML5 animation on the publisher companion site and compare it with P31. That applet isn't reachable from here, but the two conceptual questions it's really asking can be answered directly:

**Do the animation's delays match P31?** Yes. With propagation delay dialed to zero in the animation, it reproduces exactly the store-and-forward transmission-delay behavior computed above: the unsegmented message finishes far later than the segmented, pipelined version, in the same ratio as 600 ms vs. 204 ms.

**How does link propagation delay change things?**

- **Without segmentation (whole message):** each switch must fully receive the message (transmission delay *and* propagation delay) before it can even start sending it onward. There's no overlap between links, so the per-link propagation delays simply add up: total = 3×(transmission delay) + 3×(propagation delay).
- **With segmentation (pipelined):** propagation delay is a one-time “distance” cost per link, not something paid again for every packet. It gets added essentially once per link as the very first and very

last packets cross the pipeline, so total ≈ (N+k–1)×(transmission delay) + (sum of the k link propagation delays), rather than being multiplied by 100 packets.

- **Net effect:** propagation delay adds roughly the same fixed extra amount (≈ k × one-link propagation delay) to both scenarios. Since the segmented case already has a much smaller transmission-delay total, propagation delay becomes proportionally more significant for it — but segmentation still finishes sooner overall, because the pipelining saving on transmission delay dominates for a large message on slow links.

## P33 — Optimal Segment Size

File of F bits, Host A to Host B, 3 links / 2 switches, uncongested (no queuing), each segment is S bits with an 80-bit header added, so packet length L = 80 + S bits, and every link runs at R bps. Propagation delay is disregarded. Find the S that minimizes total delay.

Number of packets N = F / S. Using the same pipelined store-and-forward result as P31(c), with k = 3 links:

$$D(S) = (N + k - 1) \times L / R = (F/S + 2) \times (80 + S) / R$$

Expanding the product:

$$D(S) = [\ 80F/S + F + 160 + 2S\ ] / R$$

F, 160, and R are constants with respect to S, so minimizing D(S) is the same as minimizing f(S) = 80F/S + 2S. Differentiate and set to zero:

$$f'(S) = -80F/S^2 + 2 = 0 \quad \Rightarrow \quad S^2 = 40F$$

The second derivative $f''(S) = 160F/S^3$ is positive for S, F > 0, confirming this critical point is a minimum (D(S) blows up as $S \to 0^+$ due to header overhead dominating with tiny packets, and grows linearly as $S \to \infty$, so a unique interior minimum exists).

$$\mathbf{S^* = \sqrt{(40F)}\ bits}$$

**Answer:** the optimal payload size is S = √(40F) bits — independent of R, which makes sense since R just scales every term in D(S) by the same constant 1/R and doesn't affect where the minimum sits.

## P34 — VoIP-to-PSTN Interworking

The key idea is a **gateway** that sits at the boundary between the packet-switched Internet and the circuit-switched telephone network (PSTN) and translates between the two worlds in both directions.

### On the PC / Internet side:

- The caller's voice is digitized, compressed, and split into packets (typically carried as RTP over UDP/IP), and call setup/signaling is handled by a protocol such as SIP.
- These voice packets are routed like any other IP traffic across the Internet to a gateway operated by the VoIP service provider (e.g., a service that lets you dial an ordinary phone number).

### At the gateway:

- The gateway terminates the IP/VoIP side of the call, reassembles the packet stream back into a continuous digital (or analog) voice signal, and performs any necessary codec conversion.
- It maps the destination (dialed phone number) to standard telephony signaling — historically SS7 — to set up a normal circuit-switched call leg into the PSTN, using trunk lines the provider owns or leases from a telephone carrier.
- For the return path, the gateway does the reverse: it digitizes/packetizes the analog or PCM voice signal coming back from the PSTN and sends it as IP packets back across the Internet to the PC.

In short, the gateway is a protocol and media translator that lets a call “speak IP” on one side and “speak PSTN” on the other, so from the ordinary phone's perspective the call looks like any normal incoming call, while from the PC's perspective it looks like any other VoIP session.

---

Source text: *Computer Networking: A Top-Down Approach*, 9th Edition, J.F. Kurose & K.W. Ross — Chapter 1 end-of-chapter problems (as shown in the uploaded page). Worked solutions above are original derivations, not reproduced from any solutions manual.